

LAPORAN SOFTWARE QUALITY ASSURANCE

METODE GREYBOX – MATRIX TESTING



Anggota kelompok:

20231310046
20231310047

M. Irvan Alfiansyah
M. Nur Yanfa

PROGRAM STUDI TEKNIK INFORMATIKA
UNIVERSITAS KEBANGSAAN REPUBLIK INDONESIA

*Jln. Terusan Halimun No. 37 (Pelajar Pejuang 45) Lingkar Selatan Kec. Lengkong
kota Bandung Jawa Barat 40263*

1. DASAR TEORI MATRIX TESTING

Matrix Testing adalah metode grey box testing yang memetakan hubungan antara kebutuhan sistem, skenario pengujian, endpoint, source code, unit test, dan hasil aktual secara terstruktur dalam satu matriks.

Pendekatan ini disebut grey box karena penguji tidak hanya menguji perilaku dari luar (seperti black-box), tetapi juga menggunakan pengetahuan struktur internal kode (seperti white-box). Hasilnya adalah gambaran menyeluruh tentang kecukupan pengujian dari kedua sisi.

Pada Matrix Testing, satu skenario tidak dinilai hanya dari respons HTTP. Setiap skenario ditelusuri sampai ke:

1. Layer Handler — entry point endpoint
2. Layer Usecase — logika bisnis dan orkestrasi
3. Layer Repository — eksekusi transaksi database
4. Transaksi Database — bukti ACID
5. Test Otomatis — unit test dan k6
6. Hasil Uji Runtime — output aktual

Matrix Testing membantu menjawab pertanyaan: Apakah setiap kebutuhan checkout sudah memiliki bukti pengujian dari sisi luar (black-box) sekaligus bukti implementasi dari sisi dalam (white-box)?

2. ALASAN PEMILIHAN MATRIX TESTING

Matrix Testing dipilih sebagai tahap penggabung karena fitur Checkout ACID telah diuji secara mendalam dengan dua pendekatan terpisah sebelumnya:

Pendekatan	Bukti Pengujian Sebelumnya
Black-Box Testing	BVA, Class Partitioning, Performance Testing (k6), Endurance Testing (k6), Sample Testing
White-Box Testing	Basic Path Testing, Code Walkthrough, Data Flow Testing, Loop Testing, Formal Inspection

Matrix Testing menjadi lapisan akhir yang memastikan setiap skenario checkout memiliki hubungan yang jelas dan terdokumentasi:

Requirement → Endpoint → Source Code → Test Case → Hasil Aktual → Kesimpulan

3. RUANG LINGKUP PENGUJIAN

Pengujian berfokus pada fitur Checkout ACID. Tidak mencakup fitur non-checkout seperti katalog umum, upload gambar, admin dashboard, atau kurir, kecuali bagian supplier yang berkaitan langsung dengan order hasil checkout.

ID	Area	Dicakup	Keterangan
S-01	Checkout keranjang	Ya	Fokus utama — alur checkout end-to-end
S-02	Validasi stok	Ya	Bagian utama ACID — consistency dan isolation
S-03	Auth/JWT checkout	Ya	Mencegah checkout tanpa sesi yang valid
S-04	Payment token Midtrans	Ya	Bagian setelah order dibuat — proses eksternal
S-05	Atomicity transaksi	Ya	Pengurangan stok dan pembuatan order dalam satu unit
S-06	Performance checkout	Ya	Menggunakan hasil k6 performance test
S-07	Endurance checkout	Ya	Menggunakan hasil k6 endurance test (40 menit)
S-08	UI tampilan frontend	Tidak	Di luar scope Matrix Testing Checkout ACID

4. OBJEK SOURCE CODE YANG DIUJI

No	File	Baris	Fungsi / Bagian	Peran Dalam Matrix
1	<i>order_handler.go</i>	15-26	classifyCheckoutError	Mapping error bisnis dan gateway ke HTTP response
2	<i>order_handler.go</i>	60-87	OrderHandler.Checkout	Entry point endpoint checkout — autentikasi dan response
3	<i>order_usecase.go</i>	72-99	orderUsecase.Checkout	Orkestrasi cart, transaction, dan Snap Token
4	<i>order_repository.go</i>	23-120	CheckoutTransaction	Transaksi database ACID — inti proses checkout
5	<i>order_repository.go</i>	37, 45	Locking FOR UPDATE	Isolation transaksi stok — mencegah race condition
6	<i>order_repository.go</i>	48-64	Validasi stok	Consistency stok — cegah stok negatif
7	<i>order_repository.go</i>	86-108	Create order, items, delete cart	Atomicity dan durability — satu transaction, satu commit

5. SOURCE CODE KRITIS

5.1 Handler Checkout

Entry point endpoint checkout. Bertanggung jawab memvalidasi sesi JWT, membaca body opsional, dan menerjemahkan error usecase ke respons HTTP.

```
func (h *OrderHandler) Checkout(c *gin.Context) {
    // Validasi sesi JWT dari context middleware
    uidVal, exists := c.Get("user_id")
    if !exists {
        response.Error(c, response.ErrUnauthorized("Sesi pengguna tidak valid"))
        return
    }
    uid, ok := uidVal.(string)
    if !ok {
        response.Error(c, response.ErrUnauthorized("Sesi pengguna tidak valid"))
        return
    }

    // Body opsional - voucher_code saja
    var req struct {
        VoucherCode string `json:"voucher_code"`
    }
    _ = c.ShouldBindJSON(&req)

    order, err := h.orderUsecase.Checkout(uid, req.VoucherCode)
    if err != nil {
        response.Error(c, classifyCheckoutError(err))
        return
    }

    response.Success(c, http.StatusCreated,
        "Checkout berhasil. Silakan lakukan pembayaran.", order)
}
```

5.2 Usecase Checkout

Orkestrasi alur bisnis checkout: validasi cart, memanggil transaksi repository, dan menangani payment token Midtrans sebagai proses eksternal yang dapat gagal secara independen.

```
func (u *orderUsecase) Checkout(userID string, voucherCode string) (*domain.Order,
error) {
    cartItems, err := u.cartRepo.FindByUserID(userID)
    if err != nil {
        return nil, errors.New("gagal memuat keranjang belanja")
    }
    if len(cartItems) == 0 {
        return nil, errors.New("keranjang belanja anda kosong. tidak bisa checkout")
    }

    order, err := u.orderRepo.CheckoutTransaction(userID, cartItems, voucherCode)
    if err != nil {
        return nil, errors.New("Checkout gagal: " + err.Error())
    }

    // Payment token - proses eksternal, gagal tidak batalkan order
    snapResp, snapErr := u.snapTokenFn(order.ID, order.TotalAmount)
    if snapErr == nil && snapResp != nil {
        order.PaymentToken = &snapResp.Token
        order.PaymentURL = &snapResp.RedirectURL
        u.orderRepo.SavePaymentToken(order.ID, snapResp.Token, snapResp.RedirectURL)
    } else if snapErr != nil {
        fmt.Printf("[MIDTRANS ERROR] Gagal generate Snap Token: %v\n", snapErr)
    }

    return order, nil
}
```

5.3 Repository Transaction

Inti transaksi ACID. Seluruh operasi — lock produk, validasi stok, pengurangan stok, pembuatan order, dan penghapusan cart — dieksekusi dalam satu database transaction.

```
err := r.db.Transaction(func(tx *gorm.DB) error {
    for _, item := range cartItems {
        // Lock produk FOR UPDATE - isolation
        var product domain.Product
        tx.Clauses(clause.Locking{Strength: "UPDATE"}).
            Where("id_product = ?", item.ProductID).First(&product)

        // Validasi stok - consistency
        if product.Stock < item.Quantity {
            return fmt.Errorf("stok produk '%s' tidak mencukupi. Stok tersisa: %d",
                product.Name, product.Stock)
        }
        product.Stock -= item.Quantity
        tx.Save(&product)

        totalAmount += priceAtPurchase * float64(item.Quantity)
        orderItems = append(orderItems, domain.OrderItem{ ... })
    }

    // Atomicity: order, items, dan cart cleanup satu commit
    if err := tx.Create(&createdOrder).Error; err != nil { return err }
    if err := tx.Create(&orderItems).Error; err != nil { return err }
    if err := tx.Where("id_user = ?", userID).
        Delete(&domain.CartItem{}).Error; err != nil { return err }
    return nil // COMMIT
})
```

6. MATRIKS REQUIREMENT, ENDPOINT, SOURCE CODE, DAN TEST

Tabel berikut adalah inti dari Matrix Testing — setiap requirement dipetakan secara horizontal dari bukti black-box, bukti white-box, implementasi source code, hingga status akhir.

ID Matrix	Requirement / Skenario	Endpoint	Black-Box Evidence	White-Box Evidence	Source Code	
MX-CHK-01	Checkout berhasil membuat order	<i>POST /orders/checkout</i>	Sample ST-CHK-01; Performance success	Basic Path; Code Walkthrough	<i>OrderHandler.Checkout, CheckoutTransaction</i>	PASS
MX-CHK-02	Checkout stok kurang ditolak	<i>POST /orders/checkout</i>	BVA, Class Partitioning, Sample ST-CHK-04	Basic Path; Formal Inspection	<i>product.Stock < item.Quantity</i>	PASS
MX-CHK-03	Checkout cart kosong ditolak	<i>POST /orders/checkout</i>	Sample ST-CHK-02	Basic Path; Loop Testing	<i>len(cartItems) == 0</i>	PASS
MX-CHK-04	Checkout tanpa token ditolak	<i>POST /orders/checkout</i>	Sample ST-CHK-05	Basic Path Handler	<i>c.Get("user_id")</i>	PASS
MX-CHK-05	Error bisnis tidak menghasilkan 500/502	<i>POST /orders/checkout</i>	Sample, performance, endurance	classifyCheckout Error	<i>Error stok/cart → HTTP 400</i>	PASS
MX-CHK-06	Midtrans error tidak menghilangkan order	<i>Internal (Snap Token)</i>	White-box mock Midtrans	Basic Path; Code Walkthrough	<i>snapTokenFn, snapErr branch</i>	PASS
MX-CHK-07	Stok tidak boleh negatif	<i>POST /orders/checkout</i>	Endurance — stok akhir 0, bukan negatif	Formal Inspection	<i>Validasi stok sebelum decrement</i>	PASS
MX-CHK-08	Order dibuat dengan status PENDING	<i>POST /orders/checkout</i>	Response checkout/order	Basic Path	<i>Status: "PENDING"</i>	PASS
MX-CHK-09	Data checkout mengalir benar dari cart ke order	<i>POST /orders/checkout</i>	Black-box checkout sukses	Data Flow Testing	<i>CheckoutTransaction(userID, cartItems, voucherCode)</i>	PASS
MX-CHK-10	Banyak item cart diproses lengkap	<i>POST /orders/checkout</i>	Skenario cart multi-item	Loop Testing	<i>for _, item := range cartItems</i>	PASS

ID Matrx	Requirement / Skenario	Endpoint	Black-Box Evidence	White-Box Evidence	Source Code	
MX-CHK-11	Race condition dikendalikan	<i>POST /orders/checkout</i>	Performance/endurance tanpa 500/502	Formal Inspection	<i>clause.Locking{Strength:"UPDATE"}</i>	PASS
MX-CHK-12	CI menjaga regresi checkout	<i>GitHub Actions</i>	CI commit 22ed766 — Success	Unit test + build	<i>.github/workflows/ci.yml</i>	PASS

Seluruh 12 butir matriks (MX-CHK-01 s.d. MX-CHK-12) mendapat status PASS. Setiap requirement memiliki bukti dari kedua sisi pengujian.

7. MATRIKS ACID

Matriks berikut memverifikasi bahwa setiap prinsip ACID memiliki requirement yang jelas, implementasi source code yang dapat diidentifikasi, dan bukti test yang mendukung.

Prinsip ACID	Requirement	Source Code	Test Pendukung	Hasil
Atomicity	Stok, order, order item, dan cart cleanup harus satu transaksi	<i>r.db.Transaction(...) — semua atau rollback</i>	Basic Path, Formal Inspection	PASS
Consistency	Stok cukup sebelum dikurangi; order item memakai harga saat checkout	<i>product.Stock < item.Quantity, PriceAtPurchase</i>	Data Flow Testing, Sample Testing	PASS
Isolation	Checkout bersamaan tidak boleh menyebabkan race stok	<i>clause.Locking{Strength:"UPDATE"}</i>	Performance, Endurance, Formal Inspection	PASS
Durability	Order tersimpan permanen setelah checkout sukses	<i>tx.Create(&createdOrder), tx.Create(&orderItems)</i>	Unit test TestCheckout_Success, live checkout	PASS

8. RANCANGAN TEST MATRIX

Sepuluh test matrix berikut menggabungkan perspektif black-box (kondisi input/output) dan white-box (implementasi internal) dalam satu rancangan pengujian yang terintegrasi:

ID Test	Nama Test	Input / Kondisi	Ekspektasi	Bukti Aktual
GMT-01	Matrix checkout valid	User login, cart berisi 1 item, stok cukup	HTTP 201, order dibuat	<i>K6 ST-CHK-01, unit TestCheckout_Success</i>
GMT-02	Matrix cart kosong	User login, cart kosong	HTTP 400, tidak ada transaction	<i>K6 ST-CHK-02, unit TestCheckout_EmptyCart</i>
GMT-03	Matrix stok kurang	Quantity melebihi stok	HTTP 400/4xx valid, bukan 500	<i>K6 ST-CHK-04, unit TestCheckout_TransactionFails_StokKurang</i>
GMT-04	Matrix unauthorized	Checkout tanpa JWT	HTTP 401	<i>K6 ST-CHK-05, Basic Path handler</i>
GMT-05	Matrix Midtrans failure	Snap Token gagal	Order tetap ada, token nil	<i>unit TestCheckout_MidtransFails_OrderTetapKembali</i>
GMT-06	Matrix data flow	Cart 2 item, 1 item memiliki varian	Data tidak hilang sampai payment token	<i>unit TestDataFlow_CheckoutCartToOrderAndPaymentToken</i>
GMT-07	Matrix loop	Cart 0, 1, 3 item	Perilaku loop sesuai kondisi iterasi	<i>unit TestLoopTesting_CheckoutCartItemCounts</i>
GMT-08	Matrix endurance	Checkout durasi 40 menit	Tidak ada 500/502, stok tidak negatif	<i>K6 checkout_endurance_test.js</i>
GMT-09	Matrix performance	Checkout 10 VU	p95 < 2000ms, server error rate < 1%	<i>K6 checkout_performance_test.js</i>
GMT-10	Matrix CI regression	Push ke main branch	Backend dan frontend build lulus	<i>GitHub Actions commit 22ed766 — Success</i>

9. EKSEKUSI PENGUJIAN PENDUKUNG

9.1 Command Unit Test

```
cd backend-go
go test ./internal/usecase -run "TestCheckout|TestDataFlow|TestLoopTesting" -v
```

9.2 Output Aktual

```
=== RUN   TestCheckout_CartRepoError
--- PASS: TestCheckout_CartRepoError
=== RUN   TestCheckout_EmptyCart
--- PASS: TestCheckout_EmptyCart
=== RUN   TestCheckout_TransactionFails_StokKurang
--- PASS: TestCheckout_TransactionFails_StokKurang
=== RUN   TestCheckout_Success
--- PASS: TestCheckout_Success
=== RUN   TestCheckout_MidtransFails_OrderTetapKembali
--- PASS: TestCheckout_MidtransFails_OrderTetapKembali
=== RUN   TestCheckout_SnapRespNil_OrderTetapKembali
--- PASS: TestCheckout_SnapRespNil_OrderTetapKembali
=== RUN   TestDataFlow_CheckoutCartToOrderAndPaymentToken
--- PASS: TestDataFlow_CheckoutCartToOrderAndPaymentToken
=== RUN   TestLoopTesting_CheckoutCartItemCounts
--- PASS: TestLoopTesting_CheckoutCartItemCounts
=== RUN   TestLoopTesting_BatchProcessSupplierOrders
--- PASS: TestLoopTesting_BatchProcessSupplierOrders
=== RUN   TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder
--- PASS: TestLoopTesting_BatchProcessSupplierOrders_NoValidOrder
PASS
ok      github.com/nuryanfa/e-commerce-sqa/internal/usecase
```

Catatan: Pada TestCheckout_Success, muncul log Midtrans API key kosong karena environment test lokal tidak memakai server key production. Hal ini tidak menggagalkan test dan sesuai desain usecase — kegagalan Snap Token tidak membatalkan order internal.

10. BUKTI BLACK-BOX / K6 YANG DIPAKAI DALAM MATRIX

Jenis Pengujian	Ringkasan Hasil Aktual	Status
Checkout Performance	<i>p95=76.85ms server_error_rate=0.00% checkout_success=99 checkout_rejected=792</i>	PASS
Checkout Endurance	<i>Durasi 40 menit p95=254.35ms server_error_rate=0.00% checkout_success=140 stok akhir=0 (bukan negatif)</i>	PASS
Checkout Sample Testing	<i>Valid checkout, cart kosong, invalid quantity, overstock, unauthorized — semua sesuai ekspektasi</i>	PASS
Public Catalog Performance	<i>p95=27.04ms http_req_failed=0.00% endpoint katalog stabil</i>	PASS

11. BUKTI CI / REGRESSION GATE

CI diterapkan menggunakan GitHub Actions melalui file `.github/workflows/ci.yml`. Workflow secara otomatis menjalankan:

7. Generate Swagger docs
8. `go test ./...` — seluruh unit test backend Go
9. Build Go API binary
10. `npm ci` — install dependencies frontend
11. `npm run build` — build production frontend

Commit	Workflow	Durasi	Status
22ed766	SayurSehat CI — Backend Go + Frontend Build	1 menit 40 detik	PASS

CI ini memperkuat Matrix Testing karena hasil pengujian checkout dapat dijadikan regression gate otomatis pada setiap perubahan kode yang di-push ke main branch.

12. ANALISIS MATRIX

12.1 Kecukupan Coverage

Matrix menunjukkan bahwa semua skenario utama Checkout ACID memiliki bukti dari kedua sisi secara simultan:

- Bukti black-box berupa response endpoint, output k6, dan status HTTP.
- Bukti white-box berupa source code yang dapat ditelusuri dan unit test otomatis.

Tidak ditemukan requirement Checkout ACID utama yang hanya memiliki bukti dari satu sisi saja. Setiap butir MX-CHK-01 s.d. MX-CHK-12 memiliki kolom bukti yang terisi pada kedua sisi.

12.2 Kesesuaian Error Handling

Error bisnis seperti cart kosong dan stok kurang tidak dikembalikan sebagai 500 Internal Server Error. Error tersebut dipetakan secara eksplisit sebagai 400 Bad Request melalui fungsi `classifyCheckoutError`. Hal ini konsisten antara:

- Hasil sample testing: ST-CHK-02 dan ST-CHK-04 mendapat HTTP 400.
- Source code: `classifyCheckoutError` memetakan pesan error berbasis string ke status HTTP yang tepat.
- Performance dan endurance: `server_error_rate` = 0.00% pada seluruh skenario.

12.3 Kesesuaian ACID

Matriks ACID pada bagian 7 menunjukkan bahwa keempat prinsip memiliki bukti source code yang jelas dan dapat diverifikasi:

- Atomicity: `r.db.Transaction` — satu callback, satu commit atau rollback.
- Consistency: validasi stok sebelum decrement, `PriceAtPurchase` di-snapshot.
- Isolation: FOR UPDATE pada produk dan varian — diperkuat oleh 0% server error saat load test.
- Durability: `tx.Create` pada order dan order item — data tersimpan setelah COMMIT.

12.4 Kesesuaian Dengan Production

Pengujian performance dan endurance pernah dijalankan ke domain `production sayursehat.site`. Hasilnya menunjukkan endpoint checkout stabil dengan p95 di bawah ambang batas. Namun, pengujian live terhadap production perlu dilakukan dengan hati-hati karena dapat mengurangi stok produk yang nyata.

Rekomendasi: Siapkan produk atau environment staging khusus untuk keperluan pengujian load dan endurance agar tidak mengganggu stok production.

13. TEMUAN

ID	Temuan	Dampak	Rekomendasi	Prioritas
GMT-F01	Semua skenario Checkout ACID utama sudah punya mapping black-box dan white-box	Positif: tidak ada blind spot pengujian	Pertahankan matrix sebagai artefak traceability untuk versi berikutnya	Tinggi
GMT-F02	CI sudah berjalan sukses untuk backend dan frontend	Positif: regresi terdeteksi otomatis	Gunakan sebagai regression gate — wajib hijau sebelum merge ke main	Tinggi
GMT-F03	Live checkout testing dapat mengubah stok production	Risiko operasional — mengganggu data stok nyata	Gunakan produk khusus testing atau environment staging terpisah	Sedang
GMT-F04	Midtrans test lokal menghasilkan warning API key kosong	Tidak merusak test — hanya log warning	Gunakan mock/injection untuk Midtrans di semua environment test	Rendah
GMT-F05	Voucher tidak aktif di repository checkout, parameter masih ada	Potensi salah tafsir scope oleh pembaca kode	Dokumentasikan bahwa voucher di luar scope Checkout ACID aktif saat ini	Rendah

14. KESIMPULAN

Grey Box Matrix Testing pada fitur Checkout ACID dinyatakan **LULUS** berdasarkan 12 butir matriks (MX-CHK-01 s.d. MX-CHK-12), 10 test matrix (GMT-01 s.d. GMT-10), dan 4 prinsip ACID yang semuanya terpenuhi.

Berdasarkan matriks requirement, endpoint, source code, test otomatis, hasil k6, dan CI, seluruh skenario utama checkout memiliki bukti pengujian yang memadai dari kedua sisi:

12. Checkout berhasil — order terbentuk dengan HTTP 201.
13. Cart kosong ditolak — error bisnis, bukan server crash.
14. Stok kurang ditolak — validasi sebelum pengurangan.
15. Checkout tanpa token ditolak — autentikasi via JWT context.
16. Error bisnis tidak menjadi server crash — classifyCheckoutError memetakan ke 400/409/502.
17. Midtrans error tidak menghilangkan order internal — order tetap durable.
18. Stok tidak negatif — terbukti dari endurance test selama 40 menit.
19. Order dibuat dengan status PENDING — konsisten di seluruh pengujian.
20. Data checkout mengalir dari cart ke order dan payment token.
21. CI menjaga regression pada setiap push ke main.